

Azure Blob Storage Lifecycle Management

ARCHITECTURAL GUIDE & WORKFLOW SPECIFICATION

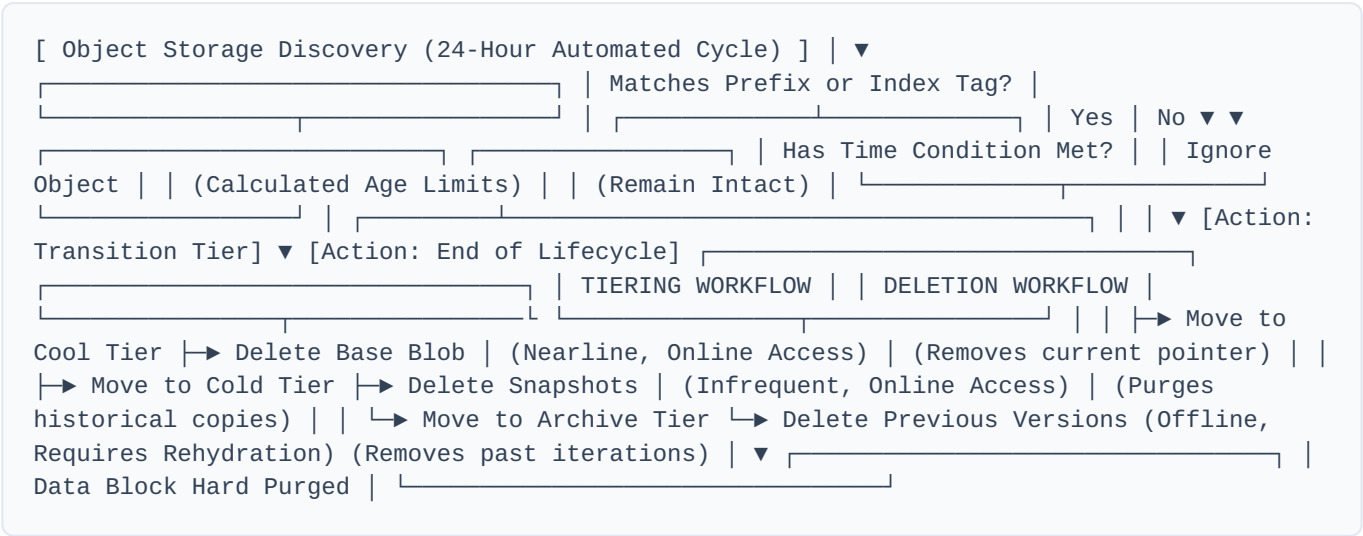
1. Structural Components of a Lifecycle Rule

An Azure Blob Storage Lifecycle Management policy operates as a built-in background engine executing automatically once every 24 hours. The primary mission of this subsystem is data tiering and retention standardization, which transforms user-defined rule matrices into declarative JSON configurations. Each rule contains three critical, nested blocks:

- Rule Details & Target Subtypes:** Defines administrative parameters including a case-sensitive alphanumeric `Rule Name`, rule execution state (`Enabled` or `Disabled`), and target file types such as Block Blobs or Append Blobs. Furthermore, it determines if actions evaluate Base Blobs, static read-only Snapshots, or historical Previous Versions.
- Filter Criteria (The Scope):** Constrains rule operations to specific objects. Without filters, a rule operates globally across the storage account. Filters allow scoping via `Blob Prefix Matches` (string matching containers or file paths like `logs/app/`) and `Blob Index Tags` (granular `Key=Value` parameters).
- Time-Based Triggers & Actions:** Dictates the exact actions executed once specific temporal bounds are satisfied. This defines the workflow pathway, moving blobs down through progressively cheaper storage tiers or purging them entirely.

2. Advanced Operational Workflow

The logic diagram below demonstrates how the automated background service inspects, routes, and dynamically shifts object storage states based on the evaluation steps inside a policy:



3. Policy Execution Constraints & Triggers

Age verification checks use three core metrics to check if structural data elements match the configured thresholds:

Days after modification check: $T_{current} - T_{modified} > X_{days}$

Days after creation check: $T_{current} - T_{created} > X_{days}$

Days after last access check: $T_{current} - T_{accessed} > X_{days}$

The access time condition requires explicitly activating `Access Time Tracking` on your storage account, which logs tracking metadata asynchronously without degrading high-throughput application I/O performance.

Storage Tier	Availability State	Billing Paradigm	Primary Target Use Case
Hot Tier	Online (Immediate)	Lowest transaction costs; highest storage fees.	Active processing workloads, production databases.
Cool Tier	Online (Immediate)	Lower storage cost; higher transaction costs.	Short-term backups, application logs, staging blocks.
Cold Tier	Online (Immediate)	Highly optimized storage cost; elevated fetch fees.	Infrequently read raw logs, analytics dumps.
Archive Tier	Offline (Hours)	Lowest absolute storage cost; high latency data rehydration.	Long-term compliance data, regulatory historical logs.
Delete Action	Extinguished	Zero billing accumulation post-purge execution.	Expired ephemeral media, temp workspaces.

Production Architecture Guardrail: When Azure Blob Versioning or Soft Delete features are active, dropping a standard base blob merely creates a historical non-current pointer. To permanently eliminate object charges, your policy rules must include explicit `snapshot` or `version` deletion workflows alongside basic blob handling.

4. Infrastructure-as-Code & Blueprint Manifests

To implement standard architectural blueprints deterministically via continuous deployment pipelines, utilize the following code deployment blocks:

Terraform HCL Manifest

```
resource "azurerm_storage_management_policy" "lifecycle_policy" {
  storage_account_id = azurerm_storage_account.example.id

  rule {
    name      = "ArchiveAndCleanupWorkflow"
    enabled   = true
    filters {
      prefix_match = ["telemetry-data/logs/", "temp-exports/"]
      blob_types   = ["blockBlob"]
    }
    actions {
      base_blob {
        tier_to_cool_after_days_since_modification_greater_than    = 30
        tier_to_archive_after_days_since_modification_greater_than = 90
        delete_after_days_since_modification_greater_than          = 365
      }
      snapshot {
        delete_after_days_since_creation_greater_than = 30
      }
    }
  }
}
```

Native Azure Resource Manager (ARM) Definition

```
{
  "policy": {
    "rules": [
      {
        "enabled": true,
        "name": "ArchiveAndCleanupWorkflow",
        "type": "Lifecycle",
        "definition": {
          "actions": {
            "baseBlob": {
              "tierToCool": { "daysAfterModificationGreaterThan": 30 },
              "tierToArchive": { "daysAfterModificationGreaterThan": 90 },
              "delete": { "daysAfterModificationGreaterThan": 365 }
            },
            "snapshot": {
              "delete": { "daysAfterCreationGreaterThan": 30 }
            }
          },
          "filters": {
            "blobTypes": [ "blockBlob" ],
            "prefixMatch": [ "telemetry-data/logs/", "temp-exports/" ]
          }
        }
      }
    ]
  }
}
```